

Software Requirements Specification (SRS)

Project Name: Modular Whole-Eye AI Diagnostic System

Version: 1.0

Date: February 2026 (Implied based on current date)

Reference Architecture: L.V. Prasad Eye Institute (LVPEI) Triage Classification

1. Introduction

1.1 Purpose

The purpose of this document is to define the software architecture, system requirements, and functional pipeline for a comprehensive, multi-model AI system designed to analyze and explain ophthalmic diseases. This SRS serves as the official blueprint for development, verification, and validation of the system.

1.2 Scope (Product Scope)

1.2.1 Identification

The system, "Modular Whole-Eye AI Diagnostic System," is an application that accepts patient symptoms (text) and optical imaging (CFP, OCT, Topography, Slit-Lamp photos). It routes this data to specialized machine learning models based on the anatomical region (Anterior vs. Posterior segment) and generates a human-readable clinical explanation.

1.2.2 Application Area

The initial scope focuses heavily on the Retina (Posterior segment) and the Cornea/Ocular Surface (Anterior segment). It is designed to assist in the preliminary diagnosis and triage of common ophthalmic conditions.

1.3 Definitions, Acronyms, and Abbreviations

Acronym/Term	Definition
SRS	Software Requirements Specification
AI	Artificial Intelligence
LLM	Large Language Model
LVPEI	L.V. Prasad Eye Institute

Acronym/Term	Definition
CFP	Color Fundus Photography
OCT	Optical Coherence Tomography
AMD	Age-Related Macular Degeneration
ResNet-50	A type of Convolutional Neural Network
F1-score	A measure of a model's accuracy (the harmonic mean of precision and recall)
FastAPI	Python web framework

1.4 References

- L.V. Prasad Eye Institute (LVPEI) Triage Classification (Reference for Router/Triage module)
- RETFound-Green Foundation Model documentation.
- PyTorch, Torchvision, Scikit-learn, and timm library documentation.

1.5 Overview of the Rest of the Document

Section 2 describes the overall architecture and general factors affecting the product.

Section 3 details the specific functional requirements (the AI modules).

Section 4 outlines external interface requirements.

Section 5 specifies the non-functional requirements (performance, security).

2. Overall Description

2.1 Product Perspective and System Architecture

The system is a self-contained AI diagnostic tool utilizing a **Hub-and-Spoke Modular AI Pipeline**. It is a standalone web application that integrates several specialized AI components.

The four distinct phases of the pipeline are:

1. **The Router (Triage):** Classifies the input (text + image type) into specific ophthalmic departments (e.g., Cornea & Anterior Segment vs. Vitreo-Retinal).
2. **The Posterior AI Spoke (Retina):** Utilizes the open-source RETFound-Green foundation model.
3. **The Anterior AI Spokes (Cornea/Surface):** Utilizes two custom-trained ResNet-50 models.

4. **The Synthesizer:** An LLM that translates the vision model output into readable patient explanations.

2.2 Product Functions (Summary of Capabilities)

The system provides the following core functions:

- Intelligent routing of uploaded ophthalmic images.
- Diagnosis of Retinal conditions (DR, Glaucoma, AMD) from CFP/OCT.
- Diagnosis and grading of Corneal Structural conditions (Keratoconus, Astigmatism) from Topography.
- Diagnosis of Ocular Surface conditions (Keratitis, Opacities) from Slit-Lamp photos.
- Generation of a plain-language clinical explanation and follow-up recommendation.

2.3 User Characteristics

User Role	Skill Level	Responsibilities/Interaction
End-Users (Patients/Clinic Staff)	Basic computer literacy	Upload eye scans and symptoms via the web interface; view results dashboard.
Administrators/Developers	Advanced (Python, ML, Cloud)	System maintenance, updating AI model weights, managing cloud hosting (Python backend).

2.4 Operating Environment

Environment Component	Details
Model Training	Google Colab (Cloud NVIDIA GPUs).
Production Backend	Python (FastAPI/Flask) hosted locally or on a cloud server.
Production Frontend	Web browser (HTML/CSS/JS) accessible via desktop or mobile (responsive design).
Hardware	Deployment Phase: CPU/GPU capable of running PyTorch inference on <code>.pth</code> weight files securely.

2.5 Constraints, Assumptions, and Dependencies

2.5.1 Constraints

- **Data Privacy:** Strict adherence to No Data Retention policy; uploaded patient images must be processed in memory and immediately discarded.
- **Infrastructure:** Vision model inference must occur on the backend server for privacy (no sending raw images to third-party APIs for vision processing).

2.5.2 Assumptions

- Input images will be of sufficient quality and correctly formatted (e.g., standard DICOM/JPEG for medical images).
- An external or local LLM API will be available and reliable for the Synthesis function.

3. Specific Requirements

3.1 Functional Requirements (Features)

The system shall perform the following functions:

3.1.1 F1: Retinal Analysis Module (Posterior Spoke)

F1.1: The system shall accept Color Fundus Photography (CFP) and Optical Coherence Tomography (OCT) images as input.

F1.2: The system shall use the RETFound-Green core model (with linear probing/transfer learning) to analyze the images.

F1.3: The system shall output a classification for the following Target Conditions: Diabetic Retinopathy, Glaucoma, and Age-Related Macular Degeneration (AMD).

3.1.2 F2: Corneal Structural Analysis Module (Anterior Spoke - Model A)

F2.1: The system shall accept Corneal Topography Maps (Pentacam/Placido disc) as input.

F2.2: The system shall use a Custom ResNet-50 Convolutional Neural Network (Model A) for analysis.

F2.3: The system shall output a grade for Keratoconus (from Normal to Severe) or classify the image as Regular Astigmatism (Control).

3.1.3 F3: Ocular Surface Analysis Module (Anterior Spoke - Model B)

F3.1: The system shall accept Slit-Lamp Photography as input.

F3.2: The system shall use a Custom ResNet-50 Convolutional Neural Network (Model B)

for analysis.

F3.3: The system shall output a diagnosis for the following Target Conditions: Infectious Keratitis (Bacterial/Fungal Ulcers), Corneal Opacities/Scarring, Pterygium, and Adjacent Conditions (Cataracts, Conjunctivitis).

3.1.4 F4: Clinical Explanation Generator (Synthesizer)

F4.1: The system shall pass the final diagnostic class (e.g., "Severe Keratoconus") from the vision models to a Large Language Model via an API prompt.

F4.2: The system shall generate and output a 3-to-4 sentence plain-language explanation of the diagnosed condition.

F4.3: The system shall generate and output a clear recommendation for clinical follow-up based on the diagnosis.

3.2 External Interface Requirements

3.2.1 User Interfaces (UI)

UI.1: The system shall provide a clean, responsive web page.

UI.2: The UI shall feature an image upload zone (`<input type="file">`).

UI.3: The UI shall include a text box for patient symptoms.

UI.4: The UI shall display a results dashboard showing the uploaded image, the detected AI diagnosis, and the generated LLM explanation.

3.2.2 Software Interfaces

SI.1: The system backend shall utilize the FastAPI (Python) framework to manage routing between the UI and the AI models.

SI.2: The system shall interface with Machine Learning Libraries including PyTorch, Torchvision, Scikit-learn, and timm.

SI.3: The system shall integrate with an external LLM (e.g., Gemini API, OpenAI API) or a local model (Llama 3) for the text synthesis function (F4).

3.2.3 Hardware Interfaces

HI.1: The system shall be deployable on hardware capable of running PyTorch inference efficiently.

4. Non-Functional Requirements

4.1 Performance Requirements

P.1 (Inference Speed): The system shall process an uploaded image, route it, run inference across the relevant model, and return the final LLM explanation within a maximum of 15 seconds.

P.2 (Model Accuracy): The custom ResNet-50 models (Model A and Model B) shall achieve an F1-score of at least 85% on validation datasets prior to deployment.

4.2 Security Requirements (See also 2.5.1 Constraints)

S.1 (Data Retention): The system shall save or log uploaded patient images to any public or persistent storage server.

S.2 (Privacy): Vision model inference must be executed locally on the backend server (on-premise or secure cloud VM) to avoid transmitting patient images to third-party vision APIs.

4.3 Maintainability

M.1: The AI model weights (.pth files) and the Python backend code shall be modularly separated to allow for independent updating of models without requiring a full system redeployment.

4.4 Portability

T.1: The production backend utilizes a Python framework (FastAPI) to ensure portability across various cloud providers or local server environments.

lar Whole-Eye AI Diagnostic System. This document serves as the official blueprint for your project, synthesizing everything we have discussed into a professional engineering format.

You can use this document to guide your development, present your project to professors or stakeholders, and keep your architecture organized.